ReAgent — Feasibility Report

HackTX 26 · NMC2 track · Oct 24, 2026 · @allen

Executive summary

Verdict: feasible, if the MVP stays to one end-to-end path. ReAgent is an open-source, self-hosted platform for building enterprise AI agents. Each agent is a declarative `.agent` file. Its settings are inherited from an enterprise layer and a team layer, and lower layers can only tighten what the layers above allow.

The MVP is a resolver and validator, a runtime built on LangChain, a dashboard with a marketplace and fork button, redacted traces, and CI-style evals run by a built-in evaluation agent. It demos two agents: a PR reviewer and a Slackbot.

Problem and track fit

The track asks teams to *"Build the platform behind the agent"*. It points to the setup work repeated for every agent (connecting tools, configuring models, writing instructions, adding monitoring) and asks for a platform that makes that setup reusable for enterprise-grade agents.

The NMC2 workshop named three specific pains:

- **Every agent starts from scratch.** Teams have no templates or prior agents to build from.
- **Connectors are a bottleneck.** A team that needs Slack must ask the AI team to build a connector, and waits.
- **SaaS is not an option.** NMC2 needs something open source it can run on its own infrastructure. Data sent to outside services is a risk, and there is no shared cloud provider.

The workshop also flagged five focus areas: connectors, evaluations, triggers, model selection, and guardrails. Security was named as a likely differentiator. NMC2 said the hackathon build doesn't have to meet its exact security requirements, but it should be built as a tool NMC2 could adopt.

NMC2 need	How ReAgent answers it
Reusable setup	`.agent` files that inherit the enterprise and team setup, and forking agents from the team's collection
Platform owner vs. builder control	Enterprise → team → agent layers that can only be tightened
Connector bottleneck	AI team registers each connector once; teams narrow it themselves
Evaluations	Evals inside every agent, rerun on each publish like CI
Model selection	Model registry served through OpenRouter; each agent picks from its team's allowed models
Guardrails and security	Secret redaction, approvals, credentials kept away from models
Observability	Traces, logs, cost, and GPU metrics per run, team, and agent
No SaaS	Self-hostable: Docker Compose for the app, any Postgres for data (Tiger Data in the demo), no SaaS required in production

Solution overview

ReAgent separates **what an agent does** (the `.agent` file) from **what it is allowed to do** (the layers above it). The runtime merges both every time the agent runs.

Core concepts

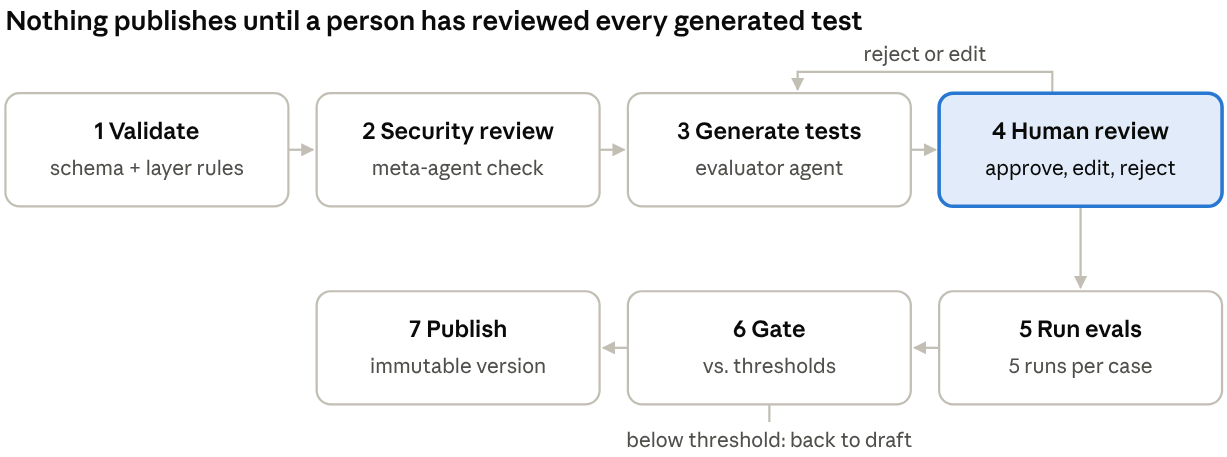
- **`.agent` file:** a declarative YAML spec (the name is final). Its fields follow the team's list: use case, tools, model, budget, ownership, evaluation, triggers, guardrails, prompt, memory, and observability, plus sub-agents, failure handling, and input/output schemas. It contains no code and no secrets, so it is safe to inspect, diff, and share.
- **Enterprise level:** the top layer, and it holds everything: every connector, model, and policy in the company, plus the enterprise prompt, budget ceilings, default evals, and enforced logging. Only enterprise admins (the AI team) can see it, or see agents that

run at this level.

- **Teams:** one flat layer under the enterprise, with no sub-teams. Each team gets a subset of the enterprise's connectors, actions, models, and budget, plus its own prompt and rules. Connectors must be granted to each team explicitly; limits are inherited. A team's agents form its collection, which members can use or fork.
- **Visibility:** there are two roles. Enterprise admins set everything about each team: members, approvers, connector grants, models, guardrails, limits, and prompt. Team members create, fork, evaluate, and run agents within those limits. Members see only their own team's collection and the connectors their team was granted; enterprise-level agent configs stay hidden from them.
- **No templates:** inheritance already gives every new agent the enterprise and team setup, so there is no separate template system. A new agent starts from a blank `.agent` file, forks an agent in the team's collection, or is drafted by the agent builder.
- **Tighten-only merge:** allowlists intersect, denylists add up, the lowest numeric limit wins, safety flags can't be turned off, and prompts are appended in order. A config that tries to loosen a rule is rejected with an error naming the field and the layer that blocked it.
- **Registry:** enterprise admins add connectors and models once. Each connector is a custom MCP wrapper we write, exposing exactly the actions in its connector file, each tagged with a risk level. Models are OpenRouter model IDs. Teams narrow what they're granted; they don't build new connectors.
- **Identity:** each permission runs either as the agent's service account or on behalf of the user who triggered it. Credentials go to the wrapper, never to the model. On behalf of a user, the agent can't do more than that user could.
- **Search:** finding agents in a team's collection runs on the desired outcome plus measured results, not just descriptions.
- **Agents calling agents:** an agent can call another as a tool: one in its own team, or an enterprise service such as `secret-scanner`. The called agent keeps its own permissions. Budget and call depth flow down, and traces and costs roll up to the caller.
- **Memory:** buckets owned by the enterprise level or a team, accessed through the same permission system. Semantic memory is facts. Episodic memory is written only by the runtime, from run logs. Procedural memory changes behavior, so a new entry applies only after the agent's evals pass again.

- **Meta-agents:** built-in agents that ship with the platform, and every user can run them from the dashboard. Their configs live at the enterprise level, so only enterprise admins can view or edit them. Each run happens on behalf of the calling user, so a meta-agent works only with that user's team, and its output never reveals enterprise-level configs.
  - **Agent builder:** drafts a new `.agent` file and its eval cases from a plain-English request, using only what the team allows. It prefers forking an existing agent.
  - **Evaluation agent:** proposes eval cases with answer keys, runs suites, and acts as the LLM judge for open-ended quality. It also grades a sample of real production runs against each agent's desired outcome, which produces the production task success rate. It is held to a human-labeled calibration set.
  - **Cost-efficiency agent:** reruns an agent's evals across model and parameter combinations and recommends the cheapest one that meets the pass-rate threshold.
  - **Security reviewer and failure triage:** the security reviewer checks every agent for least privilege before it can publish: unused permissions, unapproved write actions, broad resource scopes. The failure triage agent groups failed runs by cause and proposes a spec fix or a new eval case.

Publish pipeline (CI/CD for agents). Clicking Publish starts a pipeline, not a publish. The evaluator meta-agent generates eval tests for anything new or changed, and a team member must review every one before the evals run. A test edited after approval needs review again. Nothing publishes with unreviewed tests, and nothing publishes below threshold.



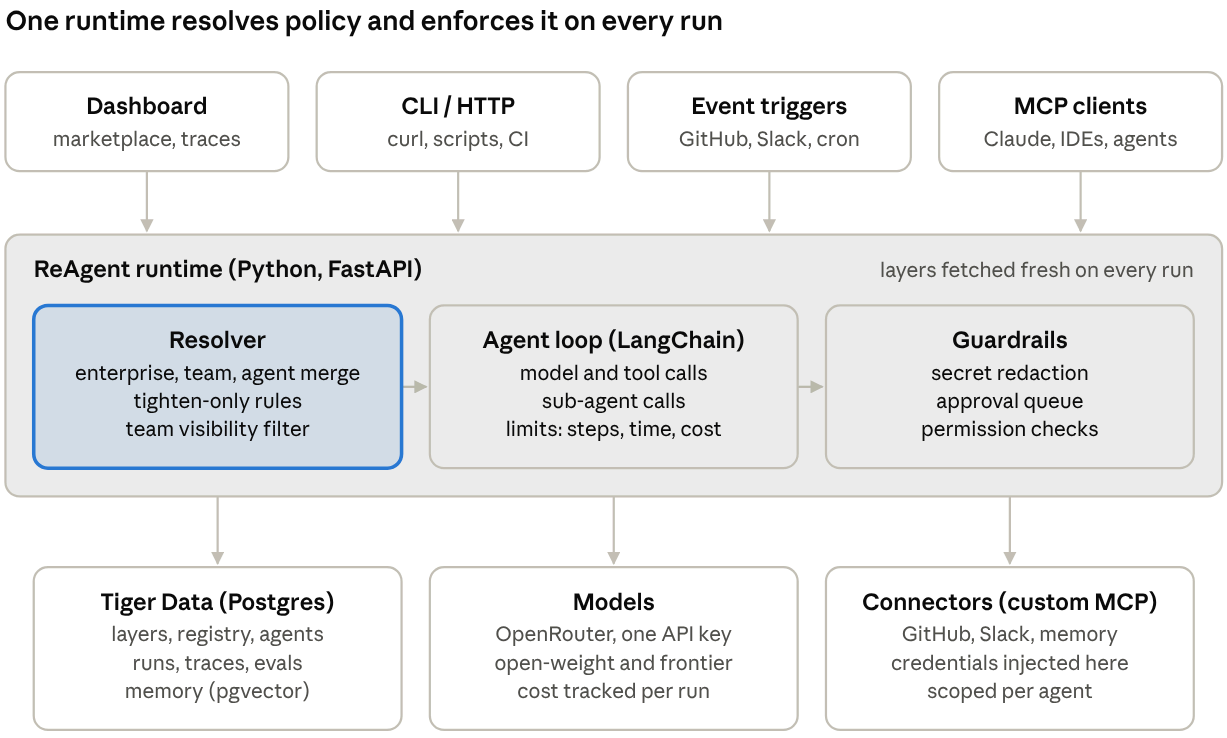
publish pipeline · 7 stages, 1 human gate

Coverage is enforced at stage 3: at least one positive, one negative, and one safety case, and every tool action exercised by some case. A **Dry run** button on each case shows real agent output during review.

The full field reference, merge rules, data model, and API are in `SPEC.md`. The `reagent.zip` bundle adds the JSON Schema, example files for the enterprise level, three teams, the registry, and seven agents, plus a validator script.

Architecture and tech stack

Every way of calling an agent goes through one runtime service. That service resolves the org and team layers fresh, runs the agent, and enforces guardrails before anything reaches a model, a connector, or a log.



ReAgent architecture · 4 entry points, 1 runtime, 3 backends

The resolver is the core: everything downstream trusts its output. Credentials live only on the connector side, so the model never sees them.

Layer	Choice	Why
Runtime and API	Python, FastAPI	LangChain is native in Python; FastAPI produces OpenAPI docs for the HTTP interface automatically
Agent engine	LangChain v1 (`create_agent`) + `langchain-mcp-adapters`	Model switching, tool calling, and middleware hooks for limits and approvals
Spec validation	JSON Schema (`jsonschema`) + Pydantic models	The schema is already written; Pydantic gives typed objects to the resolver
Database	Tiger Data (managed Postgres, Tiger Cloud free tier) with pgvector	Plain Postgres to the code; JSONB for specs, vectors for memory and search, time-series tables for runs and analytics. Also enters the Best Use of Tiger Data track
Dashboard	Next.js + React + Tailwind; Monaco for YAML editing	Decided; Monaco shows validation errors inline
Models	OpenRouter (OpenAI-compatible API)	One key for open-weight and frontier models, so no laptop GPUs are needed. Each response reports its own `usage.cost` , which feeds budgets and the cost-efficiency agent
Connectors	Custom MCP wrappers for GitHub and Slack (thin servers exposing exact action names); built-in memory and platform connectors	Matches the workshop's connector model
Tracing	Own `runs` and `run_events` tables, span-style	No extra hosted dependency; redaction happens before any write
CLI	Typer, wrapping the HTTP API	`reagent validate / run / eval / publish`

Layer	Choice	Why
Packaging	Docker Compose for runtime and dashboard; ngrok or Cloudflare Tunnel for webhooks	One command to start. In production NMC2 swaps OpenRouter for its own inference endpoint and Tiger Cloud for self-hosted Postgres; both are config changes

MVP scope

The MVP is the smallest build that tells the full NMC2 story in one demo: an admin sets policy, two teams build agents that inherit the enterprise and team setup, and the platform visibly enforces the rules. Everything below is required. If a row slips, the demo has a gap.

#	Component	MVP includes	Done when
1	Spec and validator	JSON Schema for `.agent`, team, and connector files; CLI `reagent validate`	All example files pass; a malformed file fails with a readable error
2	Resolver	Enterprise → team → agent merge with tighten-only rules, team-visibility filtering, and a resolved view showing each value's source layer	Loosening a rule fails and names the layer; an agent can't use a connector action or model its team wasn't granted
3	Runtime	LangChain agent built from a resolved spec; all models through OpenRouter; custom MCP wrapper connectors with credentials injected at the wrapper; HTTP endpoint `POST /v0/agents/{name}/runs`; a Slack mention trigger; per-run limits on steps, timeout, and budget (cost taken from OpenRouter's usage data)	The same agent runs from curl, a Python script, and the dashboard

#	Component	MVP includes	Done when
4	Guardrails	Secret redaction on outputs and traces (regex patterns plus a known-secrets list); human approval for write actions	A planted AWS key never appears in output or trace; a PR comment waits for approval
5	Sub-agent calls	One agent calls another as a tool; budget and depth flow down; nested traces	PR reviewer calls `secret-scanner`, and the trace shows a child span
6	Evals and evaluation agent	Eval engine: code checks first (tool-call trajectory, sandbox end state, secret leaks, output schema); a labeled PR benchmark of planted-defect and clean PRs reporting recall and false-alarm rate; each case run 5 times, pass rate with a confidence interval; publishing blocked below threshold. Publish runs as a pipeline: the evaluation agent generates tests, a member reviews each one (approve, edit, or reject, with a dry run), and only then do evals run. Judge calibrated against about 30 human-labeled outputs. The evaluation agent drafts cases, serves as the LLM judge, and grades a sample of real runs for production task success	A model swap shows a pass-rate and cost diff against the parent version; the evaluation agent adds a missing edge case
7	Database	Tiger Data (Postgres): enterprise level and nested teams, users and roles, registry, agents and versions, runs and events, approvals, eval runs	The dashboard reads and writes all of these

#	Component	MVP includes	Done when
8	Dashboard	Next.js + React + Tailwind: the team's collection with search and a fork button; agent editor (YAML with validation); resolved view; run trace viewer; approvals queue; a meta-agents panel every user can run (configs hidden from non-admins); enterprise admin screens; a log-in-as switcher for 3 demo users	A member can fork, edit, validate, publish, and run without the CLI, and never sees enterprise-level agents or ungranted connectors
9	Registry admin	Enterprise admins add a connector or model through a form and grant subsets of it to teams; teams submit connector requests	The Slack connector request flow works end to end
10	Demo data	Enterprise layer, 3 teams with their own collections, 4 connectors, 4–6 agents, and seeded run history	Each team's collection and the analytics don't look empty

Integrations in the MVP: GitHub (a test repo with a planted secret) and Slack (a free test workspace). All models come through OpenRouter. Use at least one cheap open-weight model and one frontier model, so the cost-efficiency comparison is real.

Stretch goals

Stretch goals are ranked by demo value per hour. Start Tier 1 only once the MVP's end-to-end path works. Tier 3 belongs in the pitch as the roadmap, not in the build. Hour figures are rough estimates for one person working with Claude Code.

Tier	Goal	Why it matters	Est. hours
1	**Agent builder (meta-agent)**	The platform builds its own agents; the strongest "wow" moment, and it answers "every agent starts from scratch"	2–3

Tier	Goal	Why it matters	Est. hours
1	**Cost-efficiency agent (meta-agent)**	Reruns evals across OpenRouter models and parameters and recommends the cheapest setup that passes; a concrete before/after number for the demo	1.5–2
1	**Mutation testing** (`reagent eval --mutants`)	Breaks an agent on purpose (tool removed, instruction deleted, weak model) and reports how many broken versions the evals catch; the strongest proof that the evals work	1–2
1	**Analytics page** (Cloudflare-style)	Runs over time, success rate, p95 latency, cost vs. budget, by team and agent, on Tiger Data time-series tables; the workshop called observability super important	2–3
1	**Every agent exposed as an MCP tool**	Calling an agent from Claude Desktop or an IDE proves "call from any stack"	1–2
1	**Policy-change impact view**	Before saving an enterprise or team change, list every agent it affects; makes live propagation visible	1–2
1	**GitHub webhook and cron triggers**	PR reviewer runs automatically on PR open	1–2
2	**Security reviewer (meta-agent)**	Checks each new agent for least privilege before it can publish	1–2
2	**Failure triage (meta-agent)**	Groups failed runs, explains the cause, and proposes a spec fix	1.5–2
2	**Upstream "update available" diff**	Forks see field-level changes in the agent they forked from and choose to merge	2
2	**Memory buckets with pgvector**	Shared semantic memory per team and enterprise; episodic memory built from run logs	3

Tier	Goal	Why it matters	Est. hours
2	**Outcome-ranked search**	Embedding search on desired outcome within a team's collection, ranked by measured eval and production success	2
2	**Trace-to-eval in one click**	Turn a good or bad real run into a test case; feeds the human intervention rate	1–2
3	Self-hosted inference in place of OpenRouter	NMC2 points the model registry at its own GPUs; GPU-hour accounting and quotas	pitch only
3	Signed, compiled `.agent` files	Governance travels with the file	pitch only
3	SSO / OIDC and Postgres row-level security	Real enterprise identity, with team visibility enforced in the database too	pitch only
3	Per-run sandboxed containers	Isolates tools that run code or shell commands	pitch only
3	Reviewed procedural memory	Agents learn procedures without skipping evals	pitch only
3	Agent Format and A2A import/export	Interoperates with the emerging standards	pitch only

Out of scope

These are deliberately not built this weekend. Each has a one-line answer ready if a judge asks.

- **NMC2's real infrastructure and security requirements.** NMC2 said not to worry about these. We also assume data confidentiality isn't a concern, so there's no data classification or model routing by sensitivity. The demo runs on our own test GitHub org, Slack workspace, and OpenRouter.
- **Production identity.** The demo uses a log-in-as switcher, not SSO. Roles are enforced in the API, but there is no real authentication.

- **Running agents outside the platform.** The tool is internal, and the runtime fetches the layers on every run, so an unregistered or copied file still gets current policy.
- **A visual no-code builder.** The users are developers, so YAML with validation plus the agent builder is the interface.
- **Multiple runtimes.** There is one Python runtime. Other stacks call it over HTTP, MCP, or the CLI.
- **Running models on our own hardware, horizontal scaling, and high availability.**
- **Full-coverage MCP servers.** Our custom wrappers expose only the actions listed in each connector file, not the whole GitHub or Slack API.

Feasibility assessment

The MVP comes to roughly **30 person-hours**, which fits a 4-person team with about 8 focused hours each, plus slack for integration. Nothing requires research or unproven technology. The risk is concentrated where components connect: the runtime, the integrations, and the dashboard. Estimates assume Claude Code does most of the typing; the team has 4 people.

Component	Est. hours	Technical risk	Main concern
Spec and validator	1	Low	Already drafted; schema and examples pass validation
Resolver	3	Low–Med	Merge rules are simple, but the rule for each field must be exactly right; unit-test every rule and every team-visibility check
Runtime (LangChain)	4	**Med–High**	Mapping spec permissions to MCP tools; credential injection; enforcing limits mid-run
Guardrails	2	Med	Redaction must wrap every output path: model output, tool results, traces, and logs
Sub-agent calls	1.5	Low–Med	Passing budget down and nesting traces

Component	Est. hours	Technical risk	Main concern
Evals	3	Med	Building the labeled PR benchmark and about 30 human calibration labels takes real time (budget an extra hour); 5 runs × cases cost OpenRouter credits
Database	1.5	Low	Standard Postgres on Tiger Cloud; JSONB plus generated columns; every query filtered by team
Dashboard	6	**Med–High**	The largest surface; polish takes longer than expected
Registry admin and connector requests	1.5	Low	Forms over the registry table
GitHub and Slack integrations	3	**High**	Writing the custom MCP wrappers, app setup, webhooks needing a public URL, and venue Wi-Fi
Demo data and seeding	2	Low	Must be realistic enough that the marketplace looks real
Total	**≈28.5**		

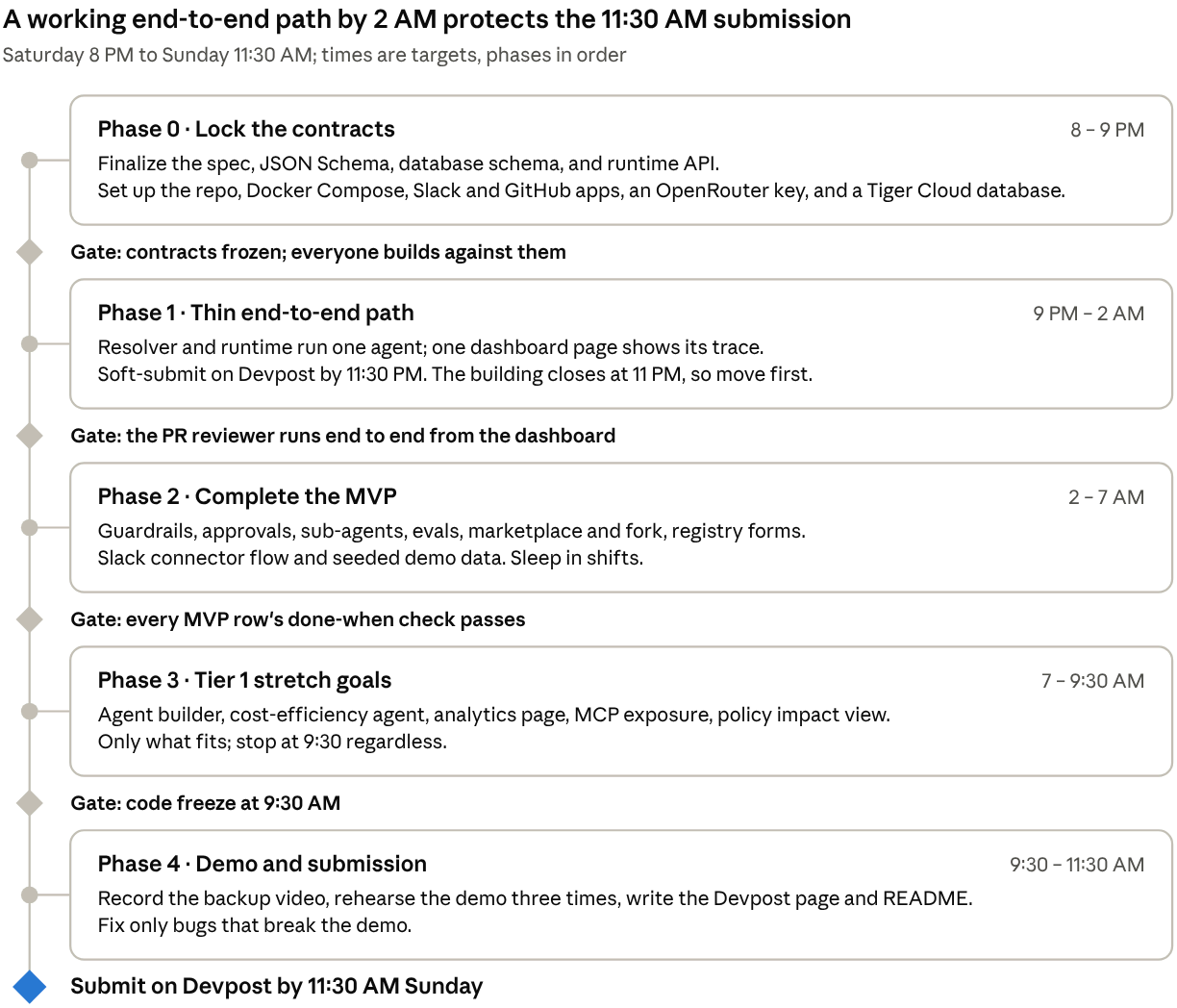
The three high-risk areas each have a fallback:

- **Runtime:** if MCP wiring stalls, call the GitHub and Slack APIs directly as LangChain tools, with the same permission checks.
- **Integrations:** use mocked connector responses (the eval `mocks` already exist) and a pre-recorded video of the live run.
- **Dashboard:** cut it to four screens: marketplace, editor with resolved view, trace, and approvals.

Build plan and timeline

The plan has five phases with a gate between each. Don't move to the next phase until its gate passes. The times assume work resumes at about 8 PM Saturday. Per the opening ceremony deck, soft submissions are due Saturday at 11:30 PM, final submissions Sunday

at 11:30 AM, and the building is closed 11 PM – 6:30 AM.



build plan · 5 phases, 4 gates

If Gate 1 slips past 4 AM, cut Phase 3 entirely and use the time to finish the MVP.

Suggested team split (4 people; roles to be decided):

Owner	Workstream	MVP rows	Phase 1 deliverable
Person A	Runtime	3, 4, 5	LangChain agent built from a resolved spec, calling one GitHub tool
Person B	Spec, resolver, evals	1, 2, 6	Resolver with unit tests for every merge rule; CLI `validate`

Owner	Workstream	MVP rows	Phase 1 deliverable
Person C	Dashboard	8	Marketplace list, agent page with resolved view, trace viewer
Person D	Data and integrations (integration owner)	7, 9, 10	Tiger Data schema, API endpoints, Slack and GitHub test apps, seed script

Risks and mitigations

The biggest risk is integration time at the end. The single most effective mitigation is a working end-to-end path by early Sunday morning, even an ugly one.

Risk	Likelihood	Impact	Mitigation
Components don't fit together late in the night	High	High	Freeze the schema and API contracts first; build a thin end-to-end path before widening anything
"One-shotting" with Claude Code produces code that only looks finished	High	High	Generate one component at a time with tests; one person owns integration
Slack or GitHub setup eats hours (OAuth, webhooks, public URL)	Med	High	Set up the apps first thing; use ngrok or a Cloudflare tunnel; keep mocked connectors as a fallback
OpenRouter credits run out, or a model is rate-limited mid-demo	Med	Med	Set a credit cap; use cheap models for eval runs and cost sweeps; give every demo agent a fallback model
Evals take too long to run live	Med	Med	Pre-run the suite; show cached results and trigger one live case

Risk	Likelihood	Impact	Mitigation
Venue Wi-Fi fails during judging	Med	High	Pre-recorded demo video; seeded data and cached runs for every screen; a phone hotspot, since OpenRouter and Tiger Cloud both need internet
Judges know Agent Format or Docker cagent and see nothing new	Med	Med	Name them in the pitch; lead with self-hosted governance, the marketplace, CI evals, and the connector flow
Redaction misses a secret on camera	Low	High	Test the exact demo PR repeatedly; layer regex patterns with an exact-match list of known demo secrets
Building closed 11 PM–6:30 AM	Certain	Med	Plan where the team works overnight before 11 PM; commit and push before moving
Scope creep into stretch goals before the MVP works	Med	High	Stretch work starts only after the MVP checklist is ticked

If a judge asks how an LLM can grade an LLM: most of the grading isn't done by an LLM. Code checks the agent's tool calls, what it actually did in the sandbox, and whether anything leaked. A labeled benchmark of planted-defect and clean PRs gives recall and false-alarm rates with a known right answer. The LLM judge covers only open-ended quality; its agreement with human labels is measured, and below the threshold it can't gate a publish. Mutation testing then shows the evals catch deliberately broken agents.

Demo plan

The demo follows one story at NMC2 and shows each feature as the answer to a specific team's problem. Target length is 4–5 minutes; the times below add up to about 5:20, so trim step 9 if needed. Team names are placeholders.

1. **The problem (20 s).** Every NMC2 team builds agents from scratch, and anyone who needs a new connector waits on the AI team.

2. **The AI team sets the enterprise level (30 s).** Log in as Frank, an enterprise admin. Show every connector and model, the enterprise prompt, secret redaction (locked), and approval for destructive actions. Grant Platform Eng a slice of the GitHub connector.
3. **Platform Eng builds from its collection (45 s).** Log in as Alice. She sees only her team's collection and connectors, never the enterprise level. She asks the agent builder for "review pull requests for bugs and leaked secrets", and it forks Code Reviewer with her repos and style guide (fallback: fork it by hand). Open the resolved view: every value is tagged enterprise, team, or agent.
4. **The rules hold (20 s).** Add the GitHub merge action to the agent; the validator rejects it, because Platform Eng was never granted merge. Try a 900-second timeout; rejected, citing the enterprise limit.
5. **Cheaper, reviewed, and proven (60 s).** Run the cost-efficiency agent. It reruns the evals on several OpenRouter models and settings, then recommends the cheapest one that still passes. Click Publish: the pipeline validates, the evaluation agent generates tests, and Alice reviews them, editing one answer key and rejecting a weak case. Only then do the evals run and the gate show the diff in pass rate, cost, and latency against v1.0.
6. **Live run (45 s).** Open a PR containing a planted AWS key. The reviewer calls `secret-scanner` as a sub-agent. The trace shows the key redacted everywhere. The review comment waits in the approvals queue; approve it, and it posts to GitHub.
7. **The connector flow (40 s).** Log in as Dana (People Ops). She can't see GitHub at all, and there's no Slack connector, so she clicks Request connector. As Frank, register Slack once with every action and grant People Ops only the channels it needs. Dana asks the agent builder for a Slackbot. It inherits People Ops' setup automatically and answers in Slack.
8. **A policy change reaches everyone (20 s).** Frank adds "no agent may read DMs." The impact view lists every affected agent, and nobody edits them.
9. **Called from anywhere (20 s).** Run the same agent from curl, a Python script, and Claude Desktop through MCP.
10. **Close (20 s).** Open source and self-hostable: NMC2 swaps OpenRouter for its own GPUs and Tiger Cloud for its own Postgres, with config changes only. Show the analytics page.

Backups: a pre-recorded video of steps 6–7, and a seeded database so every screen works without the network.

Competitive landscape

Declarative agent formats already exist, so the file format alone is not the differentiator. What none of them offers is a self-hosted platform that combines layered governance, a connector registry, a forkable marketplace, CI evals, and observability. That is exactly what NMC2 said it can't buy as SaaS.

Project	Open source	Strength	Gap ReAgent fills
Agent Format (Snap)	Yes, Apache 2.0	Org policy + agent layer with tighten-only constraints; approval gates	No public runtime (Snap Engineering); no team layer, marketplace, or evals
Docker cagent	Yes	YAML agents with MCP tools; push and pull agents via registries	Secrets in `.env` files; no layering, templates, policy, or evals
Oracle Open Agent Spec (survey)	Yes	Portable agents and flows across LangGraph, AutoGen, and CrewAI	Research stage; no security or governance model
AGENTS.md, A2A Agent Cards (survey)	Yes (Linux Foundation)	Wide adoption: instructions in markdown; signed agent identity	Not configuration: nothing enforceable about tools, budgets, or policy
LangSmith, LangGraph Platform	Hosted or licensed	Tracing, evals, deployment	SaaS by default, which NMC2 can't use; no org/team policy layers

Pitch line: *"Agent Format defines the file. We built the self-hosted platform around it that NMC2 can actually run."* Treat compatibility with Agent Format as a roadmap item. Notes on LangSmith and LangGraph Platform are from general knowledge, not a source opened for this report.

Open decisions

Most decisions are settled. Two remain open:

- [ ] **OpenRouter models and credit cap.** Pick the cheap open-weight model and the frontier model for the demo, confirm their IDs on OpenRouter, and set a spending cap.
- [ ] **Team roles.** Assign the four workstreams in the build plan.

Decided

Decision	Choice
Project name	ReAgent
Agent file name	`.agent` (YAML, with a VS Code file association)
Spec	Rewritten to the current model: see `SPEC.md` and `reagent.zip`
Layers	Enterprise → one flat team layer → agent; no sub-teams
Roles	Enterprise admin (sets every team's layer) and team member; approvers are a per-team list
Meta-agents	Agent builder, evaluator, cost optimizer, security reviewer, failure triage
Connectors	Custom MCP wrappers for GitHub and Slack
Models	OpenRouter; confidentiality assumed not to be a concern
Evaluation	Code checks and a labeled benchmark first; calibrated LLM judge only for open-ended quality; mutation testing. Publishing is a CI/CD pipeline: the evaluator generates tests and a member reviews every one before evals run
Database	Tiger Data (Postgres)
Dashboard	Next.js + React + Tailwind
Production task success	Graded by the evaluation meta-agent on a sample of real runs
Team size	4 people
Tracks	NMC2 and Best Use of Tiger Data

Sources

- HackTX 26 opening ceremony deck: NMC2 track prompt, schedule, submission deadlines, and building hours
- NMC2 workshop brainstorming notes (team's own notes)
- Agent Format and Snap Engineering: Agent Format
- Docker: Build and Distribute AI Agents and Workflows with cagent
- Zylos: Declarative Agent Specification survey (Apr 2026)
- Tiger Data: Tiger Cloud services and Agentic Postgres free plan
- OpenRouter: usage accounting